Requirements Documentation
“Charred Memory”

Bohdan Serha,Ivan Tykhomyrov,Andrii Kollomiets

Project Overview

Charred Memory is a 2D side-scrolling adventure game developed in Unity. The project explores themes of psychological horror and isolation. The narrative follows a knight who awakens with a head injury in a burnt, post-apocalyptic forest, unsure if he is in reality or a nightmare. The primary goal is to create an atmospheric experience where the player explores the environment and interacts with mysterious NPCs, specifically a watching crow.

Scope

The initial release of the project includes the following components:

- **Game Engine:** Unity 2022.3 (LTS) utilizing 2D URP.
- **Levels:** “Swamp”,”Ruined village,”Cave”
- **Characters:** One playable protagonist (Knight) and one interactive NPC (Witch).
- **Mechanics:** 2D Platforming movement, Dialogue triggers, and Scene transitions.
- **UI:** Main Menu, Pause Menu, and Settings Panel with volume control.

Functional Requirements

The system must perform the following functions:

- **Player Controller:** The system must allow the user to move the character left/right using `A/D` keys and jump using `Space`.
- **Interaction System:** The system must display a prompt (e.g., "Press E") when the player enters the trigger zone of the NPC (Witch) and initiate a dialogue sequence upon input.
- **Scene Management:** The system must preserve the player's state (e.g., specific flags) and global settings (volume) when transitioning between the "Main Menu" and "Game Level" using `DontDestroyOnLoad`.
- **Audio System:** The system must allow the user to adjust the Master Volume via a UI Slider, which updates the AudioListener in real-time.
- **Pause Functionality:** The system must freeze `Time.timeScale` when the player presses `ESC` and display the overlay menu.

Non-Functional Requirements

- **Performance:** The application must run at a stable 60 FPS on standard notebook hardware (e.g., systems with integrated graphics and 8GB+ RAM).
- **Memory Management:** The game must properly unload assets when changing scenes to prevent memory leaks, ensuring RAM usage does not spike unnecessarily (crucial for 16GB systems running background tasks).
- **Audio Compatibility:** The sound system should be compatible with spatial audio drivers (e.g., Dolby Atmos) without causing driver conflicts or muting other applications.
- **Scalability:** The code architecture should allow for the easy addition of new levels and enemy types without rewriting core systems.